BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Number of elements in the array.

Q2. First item is at index zero.

Q3. Two references identify the same object.

Q4. For a D flip-flop, D changes from 0 to 1 between active clock edges; Q was 0. Does Q change immediately? Why?

You Do pages · Independent questions

Q1. [4,6,8,12] after Insert [3]; [4,6,12] after RemoveAt [3].

Q2. Remove(5) finds first value5 [2] → [2,5,9] [1]; RemoveAt(1) removes item at index1 [2] → [2,5,9] [1]. Same final list here but different selection rule.

Q3. list.Insert(0,0) [2]; list.Add(4) [2]; Count=5 [1]; dynamic size [1].

Exit Ticket · Model answer

False. It removes the element whose index is 5.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 048 · Lesson 048 · C# List<T>: Add, Insert and Remove

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What does Array.Length give?

Q2. What is zero-based indexing?

Q3. What is aliasing between references?

Q4. 031

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

List<T> — Resizable generic sequence.

Insert — Adds at a specified index.

RemoveAt — Removes item by index.

Count — Number of items currently stored.

New learning

Observe — List<T> is a resizable sequence. Add appends; Insert inserts at a specified index and shifts following items.

Explain — RemoveAt removes an item by index; Remove(value) removes the first matching value instead.

Apply — Unlike an array, List<T> changes Count as items are inserted/removed. An index must still be within the valid bounds.

Worked example

Trace a list after inserting at the start, then removing the last element.

1. Trace and justify — After Insert: [1,2,5,7]; RemoveAt(3) removes 7; final [1,2,5].

We Do · Practise with guidance

A different list starts [3,6,9]. Add 12, then remove the VALUE 6. What remains?

```
var n=new List<int>{3,6,9};
n.Add(12);
n.Remove(6);
```

Trace and justify

[3,9,12]. Add appends 12; Remove(6) removes matching value 6, unlike RemoveAt(6).

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Trace Insert(1,6) then RemoveAt(2) for [4,8,12]. [6]

Question 2

Explain the difference between Remove(5) and RemoveAt(1) for [2,5,5,9]. [6]

Question 3

For [1,2,3], write C# that inserts 0 at start, appends 4 and outputs Count; state why List<T> suits a growing collection. [6]

Exit Ticket · Answer before checking

RemoveAt(5) means remove the value 5.